

Python for Finance - M1 Economie Finance - Partie 2

Université Paris Dauphine

L. Briens

Dauphine - CREST - Institut Polytechnique

Octobre 2024

1 Les types de variables

- Variables constantes
- Built-in variables

2 Boucles for et while, Fonctions et If

- For
- While
- Fonctions
- If

3 Les classes

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - While
 - Fonctions
 - If
- 3 Les classes

Dictionnaires

Un dictionnaire est une collection d'objet non-ordonnée. Chaque élément d'un dictionnaire est un couple clé **unique**/valeur. Il peut prendre tous les objets en valeur mais seuls les objets hashables peuvent être passés en clé. Déclaration d'un dictionnaire et affichage de ce dernier :

```
my_dict = dict([('key1', V1), ('key2', V2), ('key3', V3)])  
my_dict = {"key1": "Value1", "key2": "Value2", "key3": "Value3"}  
print(my_dict)
```

Dictionnaires

Un dictionnaire est une collection d'objet non-ordonnée. Chaque élément d'un dictionnaire est un couple clé **unique**/valeur. Il peut prendre tous les objets en valeur mais seuls les objets hashables peuvent être passés en clé. Déclaration d'un dictionnaire et affichage de ce dernier :

```
my_dict = dict([('key1', V1), ('key2', V2), ('key3', V3)])  
my_dict = {"key1": "Value1", "key2": "Value2", "key3": "Value3"}  
print(my_dict)
```

Caractéristiques

Un dictionnaire est :

- **Non-ordonné**
- **Mutable** (donc copie par référence, non-hashable)
- **Accessible (clés)**
- **Imbriquable (en valeurs uniquement)**
- **Extensible**
- **Non-hashable**

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

```
==> 889.36 889.36
```

```
print(my_dict.values())
```

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

```
==> 889.36 889.36
```

```
print(my_dict.values())
```

```
==> dict_values([133.62, 889.36, 167.23])
```

```
print(my_dict.keys())
```

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

```
==> 889.36 889.36
```

```
print(my_dict.values())
```

```
==> dict_values([133.62, 889.36, 167.23])
```

```
print(my_dict.keys())
```

```
==> dict_keys(['AMZN', 'TSLA', 'AAPL'])
```

```
print(quotes.items())
```

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

```
==> 889.36 889.36
```

```
print(my_dict.values())
```

```
==> dict_values([133.62, 889.36, 167.23])
```

```
print(my_dict.keys())
```

```
==> dict_keys(['AMZN', 'TSLA', 'AAPL'])
```

```
print(quotes.items())
```

```
==> dict_items([('AMZN', 133.62), ('TSLA', 889.36), ('AAPL', 167.23)])
```

Dictionnaires : méthodes et fonctions

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(my_dict["TSLA"], my_dict.get("TSLA"))
```

```
==> 889.36 889.36
```

```
print(my_dict.values())
```

```
==> dict_values([133.62, 889.36, 167.23])
```

```
print(my_dict.keys())
```

```
==> dict_keys(['AMZN', 'TSLA', 'AAPL'])
```

```
print(quotes.items())
```

```
==> dict_items([('AMZN', 133.62), ('TSLA', 889.36), ('AAPL', 167.23)])
```

Les 4 méthodes ci-dessus sont les moyens d'entrée dans un dictionnaire. On les manipule via ses clés (objet itérable sur ses clés), on peut le parcourir via les méthodes **.keys()**, **.values()** et **.items()**.

Dictionnaires : Suppression d'éléments

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}  
  
print(quotes.pop("AAPL", "Stocks not found"), quotes)
```

Dictionnaires : Suppression d'éléments

```
quotes = {'AMZN':133.62,'TSLA':889.36,'AAPL':167.23}
```

```
print(quotes.pop("AAPL","Stocks not found"), quotes)
```

```
==> AAPL {'AMZN': 133.62, 'TSLA': 889.36}
```

```
print(quotes.pop("AAPL","Stocks not found"), quotes)
```

Dictionnaires : Suppression d'éléments

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(quotes.pop("AAPL", "Stocks not found"), quotes)
```

```
==> AAPL {'AMZN': 133.62, 'TSLA': 889.36}
```

```
print(quotes.pop("AAPL", "Stocks not found"), quotes)
```

```
==> Stocks not found {'AMZN': 133.62, 'TSLA': 889.36}
```

```
quotes["TTE"] = 52.60
```

```
print(quotes.popitem(), quotes)
```

Dictionnaires : Suppression d'éléments

```
quotes = {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23}
```

```
print(quotes.pop("AAPL", "Stocks not found"), quotes)
```

```
==> AAPL {'AMZN': 133.62, 'TSLA': 889.36}
```

```
print(quotes.pop("AAPL", "Stocks not found"), quotes)
```

```
==> Stocks not found {'AMZN': 133.62, 'TSLA': 889.36}
```

```
quotes["TTE"] = 52.60
```

```
print(quotes.popitem(), quotes)
```

```
==> ('TTE', 52.6) {'AMZN': 133.62, 'TSLA': 889.36}
```

Dictionnaires : Suppression d'éléments

```
quotes = {'AMZN':133.62,'TSLA':889.36,'AAPL':167.23}

print(quotes.pop("AAPL","Stocks not found"), quotes)
==> AAPL {'AMZN': 133.62, 'TSLA': 889.36}

print(quotes.pop("AAPL","Stocks not found"), quotes)
==> Stocks not found {'AMZN': 133.62, 'TSLA': 889.36}

quotes["TTE"] = 52.60

print(quotes.popitem(), quotes)
==> ('TTE', 52.6) {'AMZN': 133.62, 'TSLA': 889.36}
```

Pour supprimer des éléments d'un dictionnaire la méthode **.pop()** permet de choisir la clé de l'élément à retirer et la valeur à retourner en cas d'échec. La méthode **.popitem()** retire le dernier élément ajouté au dictionnaire.

Dictionnaires : MaJ et fusions

```
stocks, prices = ['AMZN', 'TSLA', 'AAPL'], [133.62, 889.36, 167.23]
quotes = dict(zip(stocks,prices)) # tickers prix
quotes_bis = {"KO":54.51} # tickers et prix supplémentaires

all_quotes = dict(**quotes,**quotes_bis) # Python 3.5 et +
#or
all_quotes = quotes | quotes_bis # / est l'opérateur logique "ou"
print(all_quotes)
```

Dictionnaires : MaJ et fusions

```
stocks, prices = ['AMZN', 'TSLA', 'AAPL'], [133.62, 889.36, 167.23]
quotes = dict(zip(stocks,prices)) # tickers prix
quotes_bis = {"KO":54.51} # tickers et prix supplémentaires

all_quotes = dict(**quotes,**quotes_bis) # Python 3.5 et +
#or
all_quotes = quotes | quotes_bis # / est l'opérateur logique "ou"
print(all_quotes)

==> {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23, "KO":54.51}

quotes_bis.update(quotes)
print(quotes_bis.update(quotes))
```

Dictionnaires : MaJ et fusions

```
stocks, prices = ['AMZN', 'TSLA', 'AAPL'], [133.62, 889.36, 167.23]
quotes = dict(zip(stocks,prices)) # tickers prix
quotes_bis = {"KO":54.51} # tickers et prix supplémentaires

all_quotes = dict(**quotes,**quotes_bis) # Python 3.5 et +
#or
all_quotes = quotes | quotes_bis # / est l'opérateur logique "ou"
print(all_quotes)

==> {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23, "KO":54.51}

quotes_bis.update(quotes)
print(quotes_bis.update(quotes))

==> {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23, "KO":54.51}
```

Dictionnaires : MaJ et fusions

```
stocks, prices = ['AMZN', 'TSLA', 'AAPL'], [133.62, 889.36, 167.23]
quotes = dict(zip(stocks,prices)) # tickers prix
quotes_bis = {"KO":54.51} # tickers et prix supplémentaires

all_quotes = dict(**quotes,**quotes_bis) # Python 3.5 et +
#or
all_quotes = quotes | quotes_bis # / est l'opérateur logique "ou"
print(all_quotes)

==> {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23, "KO":54.51}

quotes_bis.update(quotes)
print(quotes_bis.update(quotes))

==> {'AMZN':133.62, 'TSLA':889.36, 'AAPL':167.23, "KO":54.51}
```

La méthode **.update()** permet de mettre à jour un dictionnaire de nouveaux couples clé/valeur mais aussi de mettre à jour la valeur de clés existantes. L'argument de la méthode est le dictionnaire prioritaire en cas de doublons.

Set

Un ensemble est une collection d'éléments non ordonnée, non indexée, sans doublons, mutable d'éléments non-mutables.

Déclaration d'un set et affichage de ce dernier :

```
my_set = set(['a', 'b', 'c', 'd', 'e'])  
my_set = {'a', 'b', 'c', 'd', 'e'}  
print(my_set)
```

Set

Un ensemble est une collection d'éléments non ordonnée, non indexée, sans doublons, mutable d'éléments non-mutables.

Déclaration d'un set et affichage de ce dernier :

```
my_set = set(['a', 'b', 'c', 'd', 'e'])  
my_set = {'a', 'b', 'c', 'd', 'e'}  
print(my_set)
```

Caractéristiques

Un Set est :

- **Non-ordonné**
- **Mutable** (donc copie par référence, non-hashable)
- **Accessible**
- **Non-imbriquable**
- **Extensible**

Set : méthodes et fonctions

```
my_set ={'KO', 'P911', 'FTKE', 'META'}  
my_set2 = {'KO', 'MSFT', 'MC'}  
print(my_set.add("F"), my_set)
```

Set : méthodes et fonctions

```
my_set ={'KO', 'P911', 'FTKE', 'META'}  
my_set2 = {'KO', 'MSFT', 'MC'}  
print(my_set.add("F"), my_set)  
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}  
  
print(my_set.add("F"),my_set)
```

Set : méthodes et fonctions

```
my_set ={'KO', 'P911', 'FTKE', 'META'}  
my_set2 = {'KO', 'MSFT', 'MC'}  
print(my_set.add("F"), my_set)  
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}  
  
print(my_set.add("F"),my_set)  
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}  
  
print(my_set.add(("MC.PA","MC.VI")),my_set)
```

Set : méthodes et fonctions

```
my_set ={'KO', 'P911', 'FTKE', 'META'}
my_set2 = {'KO', 'MSFT', 'MC'}
print(my_set.add("F"), my_set)
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}

print(my_set.add("F"),my_set)
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}

print(my_set.add(("MC.PA", "MC.VI")),my_set)
==> {('MC.PA', 'MC.VI'), 'F', 'META', 'KO', 'P911', 'FTKE'}
```

Set : méthodes et fonctions

```
my_set ={'KO', 'P911', 'FTKE', 'META'}
my_set2 = {'KO', 'MSFT', 'MC'}
print(my_set.add("F"), my_set)
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}

print(my_set.add("F"),my_set)
==> None {'F', 'META', 'KO', 'P911', 'FTKE'}

print(my_set.add(("MC.PA", "MC.VI")),my_set)
==> {('MC.PA', 'MC.VI'), 'F', 'META', 'KO', 'P911', 'FTKE'}
```

Les sets ne peuvent pas présenter de doublons, ils peuvent comporter des objets de types différents mais uniquement des objets hashables (immuables).

NB : Ils peuvent se montrer très utiles pour les sujets sensibles aux doublons et pour les comparaisons de dictionnaires par exemple.

Set : opérateurs logiques mathématiques

Les set permettent d'exécuter toutes les opérations usuelles sur les ensembles (intersection, union, inclusion ...).

Ci-dessous plusieurs exemples :

```
my_set2 & my_set
# or
my_set2.intersection(my_set)

my_set2 | my_set2
# or
my_set2.union(my_set)

my_set.difference(my_set2) :
my_set.isdisjoint(my_set2) :
my_set2.issubset(my_set) :
my_set.issuperset(my_set2) :
my_set2 = {'KO', 'MSFT', 'MC'}
```

Exemple

Exemples :
Dictionnaires et Sets

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - While
 - Fonctions
 - If
- 3 Les classes

Boucles for

Exemple de boucles for :

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
for stock in my_stocks:
    print (stock)

for i in range(1,10):
    print (i)

my_prices = [133.62, 889.36, 167.23, 161.11]
my_ptf = dict(zip(my_stocks, my_prices))
for stock, price in my_ptf.items():
    print (f" Le prix d\'{stock} est {price}")
```

Boucles for

Exemple de boucles for :

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
for stock in my_stocks:
    print (stock)

for i in range(1,10):
    print (i)

my_prices = [133.62, 889.36, 167.23, 161.11]
my_ptf = dict(zip(my_stocks, my_prices))
for stock, price in my_ptf.items():
    print (f" Le prix d\'{stock} est {price}")
```

Question

Quel sera l'output de ces 3 boucles ?

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - **While**
 - Fonctions
 - If
- 3 Les classes

Boucles while

Exemple de boucle While :

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
while True :
    portfolio1.append(portfolio2[-1])
    portfolio2.pop()
    if not bool(portfolio2):
        break
print(portfolio1)
```

Boucles while

Exemple de boucle While :

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
while True :
    portfolio1.append(portfolio2[-1])
    portfolio2.pop()
    if not bool(portfolio2):
        break
print(portfolio1)
```

Question

*Pourquoi le code commence par "**while True :**" ?*

Quel sera l'output ?

Boucles while

Variante plus efficiente :

```
portfolio1 = ['AMZN', 'TSLA', 'AAPL', 'META']
portfolio2 = ['V', 'TTE', 'FTKE', 'F']
while True :
    portfolio1.append(portfolio2.pop())
    if not bool(portfolio2):
        break
print(portfolio1)
```

Question

Pourquoi cette variante fonctionne-t-elle ?

Pourquoi est-elle plus efficiente/parcimonieuse ?

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - While
 - Fonctions
 - If
- 3 Les classes

Fonctions

Une fonction est une suite d'instructions que l'on peut appeler depuis l'extérieur et l'intérieur (récursivité - voir exercice factorielle TD) de la fonction.

Déclaration d'une fonction et appel de cette dernière :

Fonctions

Une fonction est une suite d'instructions que l'on peut appeler depuis l'extérieur et l'intérieur (récursivité - voir exercice factorielle TD) de la fonction.

Déclaration d'une fonction et appel de cette dernière :

```
def addition_function(a : float ,b : float = 2 ) -> float:  
    return a+b
```

```
print(addition_function (3,4))  
==> 7  
print(addition_function (3))  
==> 5
```

Fonctions

Une fonction est une suite d'instructions que l'on peut appeler depuis l'extérieur et l'intérieur (récursivité - voir exercice factorielle TD) de la fonction.

Déclaration d'une fonction et appel de cette dernière :

```
def addition_function(a : float ,b : float = 2 ) -> float:
    return a+b

print(addition_function (3,4))
==> 7
print(addition_function (3))
==> 5
```

Commentaires

Il est impératif **d'indenter** au début de la fonction.

Une bonne pratique est de préciser le type de l'input et de l'output.

Il est possible de passer un argument par défaut s'il n'est pas stipulé par l'utilisateur.

Fonctions Lambda

En cas d'un besoin ponctuel de fonction il est possible de déclarer une fonction qui ne restera pas en mémoire et ne sera plus disponible à l'appel dans la suite du code.

On appelle ce genre de fonction les **lambda functions**.

Déclaration de la fonction addition avec une fonction lambda :

Fonctions Lambda

En cas d'un besoin ponctuel de fonction il est possible de déclarer une fonction qui ne restera pas en mémoire et ne sera plus disponible à l'appel dans la suite du code.

On appelle ce genre de fonction les **lambda functions**.

Déclaration de la fonction addition avec une fonction lambda :

```
print((lambda x,y: x+y)(1,3))  
==> 4
```

Fonctions Lambda

En cas d'un besoin ponctuel de fonction il est possible de déclarer une fonction qui ne restera pas en mémoire et ne sera plus disponible à l'appel dans la suite du code.

On appelle ce genre de fonction les **lambda functions**.

Déclaration de la fonction addition avec une fonction lambda :

```
print((lambda x,y: x+y)(1,3))  
==> 4
```

Commentaires

Ce type de fonction permet d'optimiser l'utilisation de la mémoire. Les fonctions lambda sont particulièrement utiles lorsque l'on souhaite appliquer une fonction à toute une liste ou à un dataframe (voir section pandas).

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - While
 - Fonctions
 - If
- 3 Les classes

If

If sert à vérifier une ou plusieurs conditions avant d'exécuter des commandes :

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
stocks2 = ['V', 'TTE', 'AAPL', 'MS', 'AMZN']
for stocks in stocks2 :
    if stocks in my_stocks:
        print(f'{stocks} is all ready in the first portfolio')
    else :
        my_stocks.append(stocks)
print(my_stocks)
```

If

If sert à vérifier une ou plusieurs conditions avant d'exécuter des commandes :

```
my_stocks = ['AMZN', 'TSLA', 'AAPL', 'META']
stocks2 = ['V', 'TTE', 'AAPL', 'MS', 'AMZN']
for stocks in stocks2 :
    if stocks in my_stocks:
        print(f'{stocks} is all ready in the first portfolio')
    else :
        my_stocks.append(stocks)
print(my_stocks)
```

Question

Quel sera l'output de ces instructions ?

Outline

- 1 Les types de variables
 - Variables constantes
 - Built-in variables
- 2 Boucles for et while, Fonctions et If
 - For
 - While
 - Fonctions
 - If
- 3 Les classes

Créer une première classe

Définitions

"Les classes sont un moyen de réunir des données et des fonctionnalités. Créer une nouvelle classe crée un nouveau type d'objet et ainsi de nouvelles instances de ce type peuvent être construites. Chaque instance peut avoir ses propres attributs, ce qui définit son état." - [Python.org](https://python.org) - classes

Créer une première classe

Définitions

"Les classes sont un moyen de réunir des données et des fonctionnalités. Créer une nouvelle classe crée un nouveau type d'objet et ainsi de nouvelles instances de ce type peuvent être construites. Chaque instance peut avoir ses propres attributs, ce qui définit son état." - [Python.org](https://python.org) - classes

Création d'une classe simple avec une variable/attribut de classe
"marque" :

```
class Voiture :  
    """ Un exemple de classe """  
    marque = "Renault"  
  
my_car = Voiture()  
print(a)
```

Créer une première classe

Définitions

"Les classes sont un moyen de réunir des données et des fonctionnalités. Créer une nouvelle classe crée un nouveau type d'objet et ainsi de nouvelles instances de ce type peuvent être construites. Chaque instance peut avoir ses propres attributs, ce qui définit son état." - [Python.org - classes](https://www.python.org/doc/faq/classes/)

Création d'une classe simple avec une variable/attribut de classe
"marque" :

```
class Voiture :  
    """ Un exemple de classe """  
    marque = "Renault"  
  
my_car = Voiture()  
print(a)  
  
== > <__main__.Voiture object at 0x0000011940965730>  
print(a.marque)
```

Créer une première classe

Définitions

"Les classes sont un moyen de réunir des données et des fonctionnalités. Créer une nouvelle classe crée un nouveau type d'objet et ainsi de nouvelles instances de ce type peuvent être construites. Chaque instance peut avoir ses propres attributs, ce qui définit son état." - [Python.org - classes](https://www.python.org/doc/faq/classic/)

Création d'une classe simple avec une variable/attribut de classe
"marque" :

```
class Voiture :  
    """ Un exemple de classe """  
    marque = "Renault"  
  
my_car = Voiture()  
print(a)  
  
== > <__main__.Voiture object at 0x0000011940965730>  
print(a.marque)  
  
==> Renault
```

Init

Il serait plus utile de créer une classe permettant de choisir nous même le modèle de la voiture. Pour cela on utilise "init()" (built-in function) une fonction systématiquement exécutée lorsqu'un objet est instancié.

Init

Il serait plus utile de créer une classe permettant de choisir nous même le modèle de la voiture. Pour cela on utilise "init()" (built-in function) une fonction systématiquement exécutée lorsqu'un objet est instancié.

```
class Voiture :  
    def __init__(self, marque : str, couleur : str):  
        self.marque = marque  
        self.couleur = couleur  
  
my_car = Voiture()
```

Init

Il serait plus utile de créer une classe permettant de choisir nous même le modèle de la voiture. Pour cela on utilise "init()" (built-in function) une fonction systématiquement exécutée lorsqu'un objet est instancié.

```
class Voiture :  
    def __init__(self, marque : str, couleur : str):  
        self.marque = marque  
        self.couleur = couleur
```

```
my_car = Voiture()
```

```
==> TypeError: __init__() missing 2 required positional arguments
```

```
my_car = Voiture("Renault","noire")  
print(my_car.marque, my_car.couleur)
```

Init

Il serait plus utile de créer une classe permettant de choisir nous même le modèle de la voiture. Pour cela on utilise "init()" (built-in function) une fonction systématiquement exécutée lorsqu'un objet est instancié.

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

my_car = Voiture()
==> TypeError: __init__() missing 2 required positional arguments

my_car = Voiture("Renault","noire")
print(my_car.marque, my_car.couleur)
==> Renault noire
```

On peut désormais instancier plusieurs objets Voiture de marque différente et de couleur différente avec la même classe.

Méthodes de classe (1/3)

Les classes sont dotés d'attributs (ici marque et couleur) mais peuvent aussi être dotées de méthodes (fonction propre à une classe).

```
class Voiture :  
    def __init__(self, marque : str, couleur : str):  
        self.marque = marque  
        self.couleur = couleur  
  
    def repaint(self, color : str):  
        self.couleur = color  
  
my_car = Voiture("Renault", "noire")  
my_car.repaint("vert")  
print(my_car.couleur)
```

Méthodes de classe (1/3)

Les classes sont dotés d'attributs (ici marque et couleur) mais peuvent aussi être dotées de méthodes (fonction propre à une classe).

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

    def repaint(self, color : str):
        self.couleur = color

my_car = Voiture("Renault","noire")
my_car.repaint("vert")
print(my_car.couleur)

==> vert
```

Méthodes de classe (1/3)

Les classes sont dotés d'attributs (ici marque et couleur) mais peuvent aussi être dotées de méthodes (fonction propre à une classe).

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

    def repaint(self, color : str):
        self.couleur = color

my_car = Voiture("Renault", "noire")
my_car.repaint("vert")
print(my_car.couleur)

==> vert
```

Ici repaint() est une méthode avec un argument : "color".

NB : Le "self" en premier argument de la méthode n'est pas un argument à renseigner par l'utilisateur. Le self sert à préciser que cette méthode s'applique à l'instance.

Méthodes de classe (2/3)

Ce code est strictement équivalent au précédent :

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

    def repaint(self, new_color : str):
        self.couleur = new_color

Voiture.repaint(my_car, "vert")
print(my_car.couleur)
==> vert
```

La méthode à était appelée depuis l'objet class Voiture, et non plus depuis l'instance de classe my_car, ainsi on a eu à renseigner le "self" comme étant l'instance "my_car".

Méthodes de classe (3/3)

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

    def open_door(self):
        return "Door opened"

my_car = Voiture("Renault","noire")
method_open = my_car.open_door
print(method_open())
==> Door opened

print(type(method_open), type(method_open()))
```

Méthodes de classe (3/3)

```
class Voiture :
    def __init__(self, marque : str, couleur : str):
        self.marque = marque
        self.couleur = couleur

    def open_door(self):
        return "Door opened"

my_car = Voiture("Renault","noire")
method_open = my_car.open_door
print(method_open())
==> Door opened

print(type(method_open), type(method_open()))
==> <class 'method'> <class 'str'>
```

On remarque que la méthode de classe doit être munie des "()" pour être appelée, elle peut être instanciée dans une autre variable et exécutée à tout moment et autant de fois que nécessaire par la suite.

Attribut de classe VS attribut d'instance (1/3)

L'utilisation d'attributs de classe à la place d'attributs d'instance est dangereuse, en effet une modification d'un attribut de classe peut se propager à toutes les instances de la classe sous certaines conditions !

```
class Voiture :
    puissance = 100 # partagé par toutes les instances de la classe
    def __init__(self, marque : str, couleur : str):
        self.marque = marque # propres à chaque instances
        self.couleur = couleur

    def add_power(self, extra_power : int):
        self.puissance += extra_power

my_black_car = Voiture("Renault", "noire")
my_red_car = Voiture("Renault", "rouge")
my_black_car.add_power(50)
print(my_black_car.puissance, my_red_car.puissance)
```

Attribut de classe VS attribut d'instance (1/3)

L'utilisation d'attributs de classe à la place d'attributs d'instance est dangereuse, en effet une modification d'un attribut de classe peut se propager à toutes les instances de la classe sous certaines conditions !

```
class Voiture :
    puissance = 100 # partagé par toutes les instances de la classe
    def __init__(self, marque : str, couleur : str):
        self.marque = marque # propres à chaque instances
        self.couleur = couleur

    def add_power(self, extra_power : int):
        self.puissance += extra_power

my_black_car = Voiture("Renault", "noire")
my_red_car = Voiture("Renault", "rouge")
my_black_car.add_power(50)
print(my_black_car.puissance, my_red_car.puissance)
```

==> 150 100

Attribut de classe VS attribut d'instance (2/3)

Ci-dessous le même type d'exemple qu'à la slide précédente mais l'attribut de classe est une liste :

```
class Voiture :
    broken_parts = [] # partagé par toutes les instances de la classe
    def __init__(self, marque : str, couleur : str):
        self.marque = marque # propres à chaque instances
        self.couleur = couleur

    def append_broken_parts(self, new_broken_part : str):
        self.broken_parts.append(new_broken_part)

my_black_car = Voiture("Renault","noire")
my_red_car = Voiture("Renault","rouge")
my_black_car.append_broken_parts("breaks")
print(my_black_car.broken_parts,my_red_car.broken_parts)
```

Attribut de classe VS attribut d'instance (2/3)

Ci-dessous le même type d'exemple qu'à la slide précédente mais l'attribut de classe est une liste :

```
class Voiture :
    broken_parts = [] # partagé par toutes les instances de la classe
    def __init__(self, marque : str, couleur : str):
        self.marque = marque # propres à chaque instances
        self.couleur = couleur

    def append_broken_parts(self, new_broken_part : str):
        self.broken_parts.append(new_broken_part)

my_black_car = Voiture("Renault","noire")
my_red_car = Voiture("Renault","rouge")
my_black_car.append_broken_parts("breaks")
print(my_black_car.broken_parts,my_red_car.broken_parts)

==> ['breaks'] ['breaks']
```

Que remarque-t-on ?

Attribut de classe VS attribut d'instance (3/3)

Explications

Lorsqu'un attribut de classe est un objet de type "non-hashable" alors une modification de ce dernier se propage à toutes les instances existantes de cette classe. En effet, un objet non hashable est copié en référence, une modification de l'un entraîne modification de tous.